

PROJECT DOCUMENTATION

WAF Demo Manager

A hands-on lab that shows how a Web Application Firewall (ModSecurity + OWASP Core Rule Set) detects and blocks attacks against a deliberately vulnerable web application.

VERSION	**1.0**
AUTHOR	**www.tca.ac.tz**
DOCUMENT DATE	**October 2026**
TARGET AUDIENCE	**Instructors, students, lab implementers**

FOR AUTHORIZED SECURITY TRAINING USE ONLY

OWASP Juice Shop is intentionally vulnerable. Run this lab only on an isolated virtual network that you control, never on a production or shared network.

Contents

1. About this document . . . 5
1.1 Who should read what . . . 5
1.2 Conventions used . . . 5
1.3 Source of truth . . . 5

2. Executive summary . . . 6
2.1 The problem it solves . . . 6
2.2 What it is, in plain words . . . 6
2.3 What the audience sees . . . 6
2.4 What you need . . . 6

3. Background: the key ideas . . . 7
3.1 Web applications and why they get attacked . . . 7
3.2 What a Web Application Firewall (WAF) and a reverse proxy are . . . 7
3.3 ModSecurity and the OWASP Core Rule Set . . . 7
3.4 Anomaly scoring: how the decision is made . . . 8
3.5 Detection-only versus blocking . . . 8
3.6 Paranoia level and false positives . . . 8
3.7 Docker and containers (the minimum you need to know) . . . 8

4. Architecture . . . 9
4.1 Topology . . . 9
4.2 What happens to one request . . . 10
4.3 Why the shop has no published port . . . 10
4.4 The two phases of use . . . 10

5. Requirements . . . 12
5.1 Hardware (recommended) . . . 12
5.2 Software . . . 12
5.3 Network . . . 12
5.4 Fully offline option . . . 12
5.5 Permissions . . . 12

6. Implementation guide . . . 13
6.1 Overview of the steps . . . 13
6.2 Create the isolated network (VirtualBox example) . . . 13
6.3 Create VM 2 (Server) . . . 13
6.4 Create VM 1 (Attacker) . . . 14
6.5 Copy the script to VM 2 . . . 14
6.6 Run the script for the first time . . . 14
6.7 Verify the isolation . . . 15
6.8 Before the demonstration: remove the NAT adapter . . . 15

7. Using the menu . . . 17

7.1 Option 1: Setup . . . 17
7.2 Option 2: WAF protection . . . 18
7.3 Option 3: Live view & logs . . . 18
7.4 Option 4: Guided demo . . . 18
7.5 Option 5: Tools . . . 19

8. How the script works internally . . . 20
8.1 Design principles . . . 20
8.2 Code map . . . 20
8.3 Startup and sudo elevation . . . 21
8.4 Automatic Docker installation and repair . . . 21
8.5 Getting the images . . . 22
8.6 The generated compose file . . . 23
8.7 State and files on disk . . . 23
8.8 What Setup does, step by step . . . 23
8.9 Switching protection ON or OFF . . . 24
8.10 Port selection and firewall . . . 24
8.11 The log pipeline . . . 24
8.12 The guided demo, in detail . . . 25
8.13 Self-repair behaviour . . . 26
8.14 User interface behaviour . . . 26

9. Running the demonstration . . . 27
9.1 Room and screen layout . . . 27
9.2 Timeline and talk track . . . 27
9.3 Presenter tips . . . 28
9.4 If something goes wrong during the demo . . . 28

10. Command reference . . . 29
10.1 Starting the script . . . 29
10.2 Command-line options . . . 29
10.3 The --run actions . . . 29
10.4 Attacker commands and expected results . . . 30
10.5 Reading the results . . . 31

11. Troubleshooting . . . 32
11.1 Installing the images offline . . . 32
11.2 Collecting information for a bug report . . . 33

12. Security and ethics . . . 34
12.1 Isolation rules . . . 34
12.2 Safety built into the project . . . 34
12.3 What to know about the script itself . . . 34
12.4 Cleaning up . . . 34

13. Limitations, known issues and items to verify . . . 35
13.1 Limitations (by design) . . . 35

13.2 Items to verify on your own lab ("To be verified") . . . 35

14. Glossary and FAQ . . . 36

14.1 Glossary . . . 36

14.2 Frequently asked questions . . . 37

Appendix A — File and folder layout . . . 38

Appendix B — The generated compose file and its variables . . . 39

Appendix C — Complete menu map . . . 40

Appendix D — Attack types and CRS rule families . . . 41

Appendix E — Post-demo cleanup checklist . . . 42

Appendix F — Pre-demo checklist (printable) . . . 43

Quick-start card . . . 44

1. About this document

This document explains the **WAF Demo Manager** project completely: what it is, why it exists, how it works inside, and how to build and run it yourself. You do not need any prior knowledge of firewalls, Docker or web security to follow it.

1.1 Who should read what

The document is written for three kinds of reader. Each reader can use a different path through it.

Reader	You are...	Read these sections
A Non-technical	a manager, teacher or student who wants to understand the idea and what the demo shows	2 Executive summary, 3 Background (first half), 9 Running the demonstration (read the table of expected results), 14 FAQ
B Instructor / presenter	the person who will run the demo in front of an audience	2, 4 Architecture, 7 Using the menu, 9 Running the demonstration, 10 Command reference, 11 Troubleshooting, Appendix F (pre-demo checklist) and the Quick-start card at the end
C Technical implementer	the person who builds the lab and wants to understand every part of the script	4, 5 Requirements, 6 Implementation guide, 8 How the script works, 11, 12 Security, 13 Limitations, all appendices

1.2 Conventions used

- Commands and file names appear in `this style`. Code and terminal output appear in dark boxes.
- Under every command you will find an **Expected result** line. If you see something different, go to section 11 (Troubleshooting).
- **WARNING** marks something that can cause damage or break the isolation of the lab. **TIP** marks a shortcut. **TO BE VERIFIED** marks a statement that could not be confirmed from the source code alone and should be checked on your own lab.
- Example outputs are illustrative: your times, IP addresses and numbers will differ.

1.3 Source of truth

All facts about behaviour in this document come from the single source file **waf_demo_manager.py** (version 1.0, author www.tca.ac.tz, about 1,700 lines of Python, standard library only). If this document and the script ever disagree, the script is correct.

> **NOTE:** Development testing of the script used a simulated Docker environment (menus, setup, mode switching, guided demo, log views, reset). The project owner has also run it on a real lab VM. The log-reading code was corrected after that run, so its output on a real WAF should be re-confirmed. Section 13 lists every item that still needs confirmation on real hardware.

2. Executive summary

WAF Demo Manager is a one-file tool that builds a small, safe lab in which an audience can watch a **Web Application Firewall (WAF)** stop real attacks, and can also see what a WAF cannot stop.

2.1 The problem it solves

Most people have heard that a WAF "protects websites", but few have seen what it actually does to a request. Slides and diagrams rarely convince an audience. This project lets people watch the same attack **succeed** when the firewall is only watching, and **fail with an error 403** when the firewall is blocking, while a live screen shows who attacked, with what kind of attack, and what the firewall decided.

2.2 What it is, in plain words

- A **vulnerable web shop** (OWASP Juice Shop) that is deliberately full of security holes. It is the victim.
- A **firewall for web applications** (ModSecurity with the OWASP Core Rule Set) placed in front of the shop. Every visitor must pass through it. It is the guard.
- A **Python script** (`waf_demo_manager.py`) with a simple coloured menu that installs both pieces automatically and lets the presenter switch the guard between *watching* and *blocking*, and see its logs.
- A second machine, the **attacker** (Kali Linux), from which the audience sees the attacks being sent.

2.3 What the audience sees

Attack	Guard only WATCHES (protection OFF)	Guard BLOCKS (protection ON)
SQL injection (login bypass and search)	Reaches the shop. Logged.	Error 403. The shop never sees it.
Cross-site scripting	Reaches the shop. Logged.	Error 403.
Path traversal	Reaches the shop. Logged.	Error 403.
Command injection	Reaches the shop. Logged.	Error 403.
Automatic scanner (sqlmap / nikto style)	Reaches the shop. Logged.	Error 403, usually very quickly.
Business-logic flaw (tampering with another user's basket)	Reaches the shop.	**Still reaches the shop.** The request looks perfectly normal, so the WAF cannot see a problem.

The last row is the most important lesson: **a WAF is one layer of defence, not a complete defence.** Flaws in the application's own logic must be fixed in the application.

2.4 What you need

- One computer that can run two virtual machines (VirtualBox, VMware, Proxmox or KVM).
- VM 2 "Server": Ubuntu or Debian. This is where the script runs.
- VM 1 "Attacker": Kali Linux (or any Linux with curl, sqlmap and nikto).
- A temporary internet connection for VM 2 during preparation (to download Docker and two container images). The demonstration itself runs on an isolated network with no internet.
- Time: building the VMs takes longest. **Setup** then takes a few minutes, mostly image download time. A full demonstration takes about 30 to 40 minutes.

3. Background: the key ideas

This section explains every technical term the rest of the document relies on. Read it once; refer back to the glossary (section 14) later.

3.1 Web applications and why they get attacked

A **web application** is a program that runs on a server and answers requests sent by browsers over HTTP: show a page, log a user in, search a catalogue. Every request carries text the user typed or chose: a search word, an e-mail address, a file name. Attackers hide malicious instructions inside that text. If the application trusts it blindly, the instructions run.

Attack	Plain-language explanation	Typical sign in the request
SQL injection (SQLi)	The attacker's text is treated as a database command, so they can log in without a password or read other users' data.	`' OR 1=1--`, `UNION SELECT`
Cross-site scripting (XSS)	The attacker's text contains a script that runs in another visitor's browser.	`<script>alert(1)</script>`
Path traversal	The attacker climbs out of the intended folder to read system files.	`../../etc/passwd`
Command injection	The attacker's text is passed to the operating system as a command.	`;cat /etc/passwd`
Automated scanning	A tool such as sqlmap or nikto fires thousands of probes to find weaknesses.	tool name in the User-Agent header, high request volume
Business-logic flaw	The request is perfectly well-formed, but the application does the wrong thing with it, e.g. lets user A change user B's basket.	nothing unusual: looks like a normal request

3.2 What a Web Application Firewall (WAF) and a reverse proxy are

A **WAF** sits between the visitor and the web application and inspects every HTTP request (and optionally the response). If a request looks like an attack, the WAF can log it, block it, or both. Unlike a network firewall, which looks at addresses and ports, a WAF reads the **content** of web requests. A WAF protects *against known attack patterns in the request*; it does not repair the application.

A **reverse proxy** accepts requests on behalf of another server and forwards them. In this lab the visitor believes he is talking to the shop but is really talking to the WAF, which forwards allowed requests to the shop and hands the answer back. Because it relays every request, it is the ideal place to inspect them.

3.3 ModSecurity and the OWASP Core Rule Set

- **ModSecurity** is the open-source WAF engine. Here it runs inside **nginx** (a web server), which acts as the reverse proxy.
- The **OWASP Core Rule Set (CRS)** is a free collection of detection rules written by the OWASP community. Each rule has a numeric ID; the first three digits tell which family it belongs to (for example 942xxx = SQL injection, 941xxx = cross-site scripting). The script uses these ranges to name each attack in plain words (Appendix D).
- **OWASP Juice Shop** is a deliberately insecure web shop made for security training. Nothing in it is real data.

3.4 Anomaly scoring: how the decision is made

CRS does not block on the first suspicious word. Each matching rule adds **points** to the request's **anomaly score**, depending on its severity (critical rules add 5, error 4, warning 3, notice 2). When the total reaches the **threshold**, the request is considered an attack. In this lab the inbound threshold is **5** (setting `ANOMALY_INBOUND`), the CRS default, so a single critical match is enough.

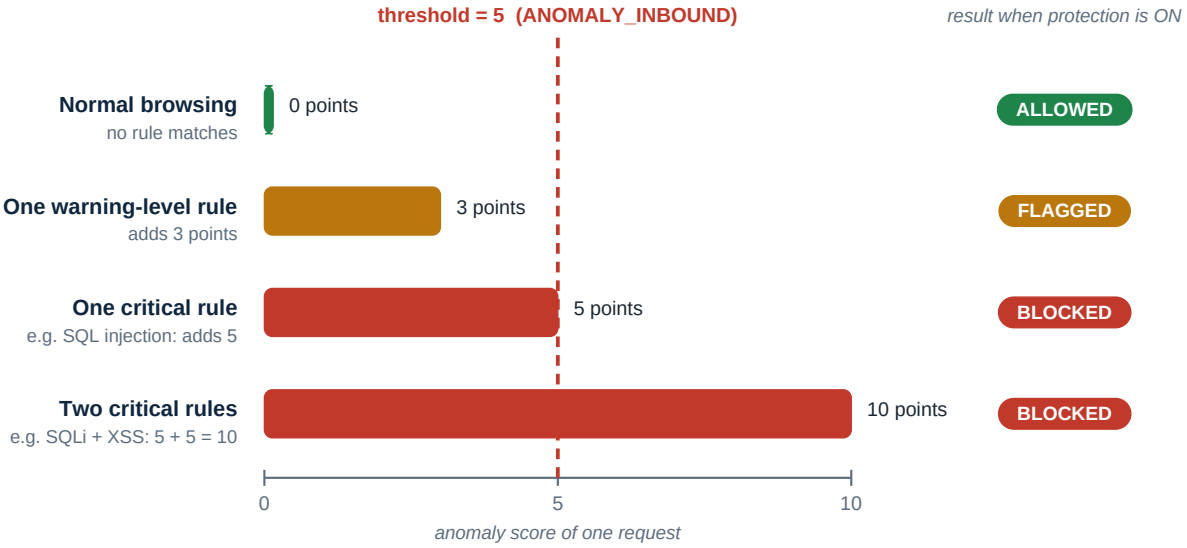


Figure 1. *How the anomaly score decides the outcome (threshold 5). With protection OFF, a request at or above the threshold is only logged; with protection ON it is blocked.*

3.5 Detection-only versus blocking

The same rules run in both modes. The only difference is what happens when the threshold is reached:

	Protection OFF	Protection ON
ModSecurity setting	`DetectionOnly`	`On`
Attack request is...	forwarded to the shop (the attack works)	stopped at the WAF
Attacker receives	the shop's normal answer, e.g. 200	403 Forbidden, sent by the WAF
Event is logged?	Yes. The log marks it as "would have been blocked"	Yes, marked blocked

3.6 Paranoia level and false positives

CRS has four **paranoia levels**. Level 1 (used by default here) catches well-known attacks with very few false alarms. Levels 2 and 3 enable stricter rules: they catch more, but they also start blocking **legitimate** requests (a **false positive**), for example a search term with unusual characters. The script offers "Stricter mode" so a presenter can show this trade-off. A real operator would then add a targeted exclusion instead of lowering protection everywhere.

3.7 Docker and containers (the minimum you need to know)

- A **container** is an isolated package that runs one program with everything it needs; an **image** is the template it is created from. **Docker Compose** starts several containers from one file (`docker-compose.yml`, written by the script).
- A **published port** makes a container reachable from outside the machine. A container with **no published port** can only be reached by other containers on the same private Docker network. This lab relies on exactly that to force all traffic through the WAF.

4. Architecture

Two virtual machines on one isolated virtual network. The attacker can only reach the shop through the WAF.

4.1 Topology

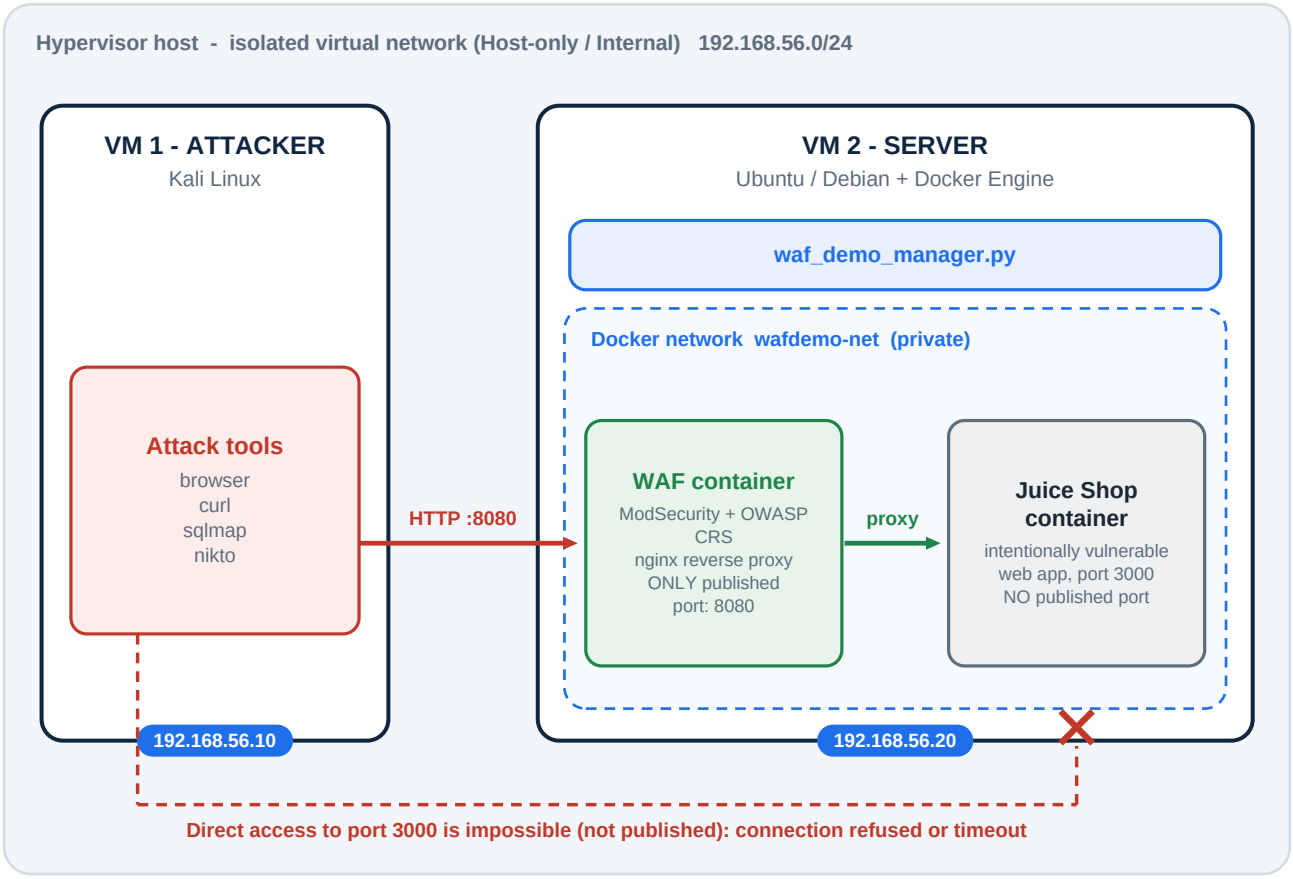


Figure 2. *Lab topology. IP addresses are examples; use any private subnet that you control.*

Component	Role	Details
Hypervisor host	Runs both VMs	VirtualBox, VMware, Proxmox or KVM. Provides the isolated virtual network.
VM 1 - Attacker	Sends the attacks	Kali Linux (or any Linux with `curl`, `sqlmap`, `nikto` and a browser). Example IP 192.168.56.10.
VM 2 - Server	Runs the lab	Ubuntu/Debian with Docker Engine and Docker Compose v2. Runs `waf_demo_manager.py`. Example IP 192.168.56.20.
Container `wafdemo-waf`	The guard	Image `owasp/modsecurity-crs:nginx` (ModSecurity + CRS inside nginx). The **only** published port: 8080 (default). Forwards to `http://juice-shop:3000`.
Container `wafdemo-juice-shop`	The victim	Image `bkimminich/juice-shop`. **No published port**; reachable only from the WAF over the private network `wafdemo-net`.
`waf_demo_manager.py`	The controller	Installs, configures and observes the two containers; never sits in the traffic path.

4.2 What happens to one request

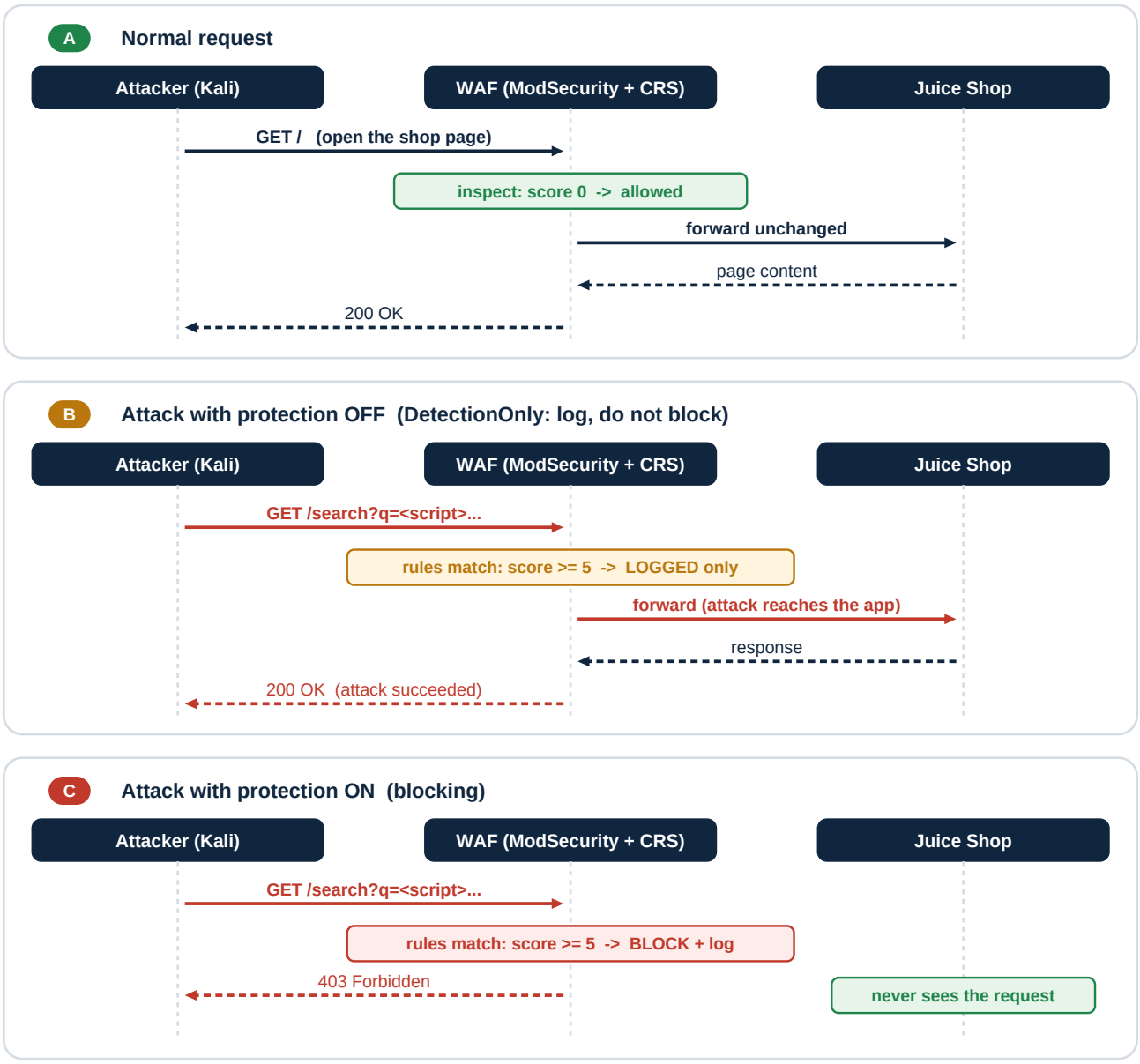


Figure 3. *The life of a request: (A) a normal request is forwarded and answered; (B) an attack with protection OFF is logged but still reaches the application; (C) an attack with protection ON is stopped by the WAF.*

4.3 Why the shop has no published port

If the shop published its own port, an attacker could skip the WAF and attack it directly, and the demonstration would prove nothing. By publishing **only** the WAF's port, the WAF becomes "the only door in". The script verifies this condition: the health check in menu 5 reports an isolation failure if the shop container ever publishes a port.

4.4 The two phases of use

The lab needs internet only once, to download software. After that it runs fully offline on an isolated network.

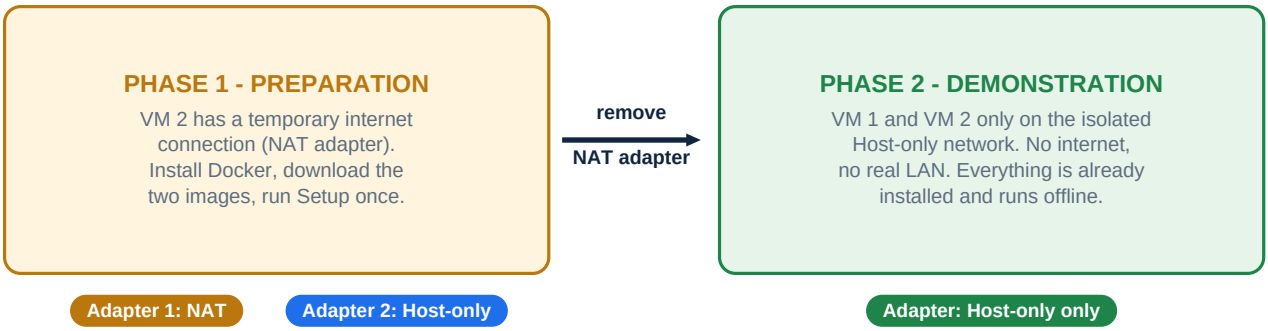


Figure 4. Preparation needs a temporary internet connection (NAT adapter on VM 2). The demonstration runs on the isolated Host-only network only.

WARNING: Never leave either VM on a **Bridged** or **NAT** network during the demonstration. Juice Shop is intentionally vulnerable and the attacker VM contains attack tools; both must stay on the isolated network.

5. Requirements

5.1 Hardware (recommended)

The script itself only warns when VM 2 has **less than 4 GB free disk** or **less than 1 GB free memory**. The sizes below are practical recommendations, not limits enforced by the script.

Resource	Host computer	VM 2 (Server)	VM 1 (Attacker)
CPU	4 cores or more	2 vCPU	2 vCPU
Memory	8 GB or more	4 GB (script warns below 1 GB free)	2 GB
Disk	40 GB free	20 GB (script warns below 4 GB free)	20 GB

5.2 Software

Where	Software	Notes
Host	VirtualBox, VMware, Proxmox or KVM	Any hypervisor that supports an isolated (host-only / internal) network.
VM 2	Ubuntu Server or Debian	The script installs Docker with `apt` (Debian/Ubuntu family). Other Linux distributions rely on the official Docker installer script as a fallback.
VM 2	Python 3.7 or newer	Present by default on current Ubuntu/Debian. No extra Python packages are needed (standard library only).
VM 2	Docker Engine + Compose v2	Installed automatically by the script if missing.
VM 1	Kali Linux	Includes `curl`, `sqlmap`, `nikto` and a browser.

5.3 Network

- One **Host-only** (VirtualBox) or **Internal** network shared by both VMs, with static IP addresses in the same subnet.
- For the preparation phase only: a second **NAT** adapter on VM 2 so it can reach the internet.
- Outbound access from VM 2 to Docker Hub (`registry-1.docker.io`) or `ghcr.io` on port 443, and to the Ubuntu/Debian package mirrors, during preparation.

5.4 Fully offline option

If VM 2 can never reach the internet, install Docker from an offline package source and place the two image files in `/opt/waf_demo/images/` as `.tar` archives. The script loads them automatically (section 11 shows the commands).

5.5 Permissions

Installing Docker and starting containers needs administrator rights. You do not have to type `sudo` yourself: the script re-launches itself with `sudo` when it is not running as root.

6. Implementation guide

Follow these steps in order. Every step has a check so you know it worked before you continue.

6.1 Overview of the steps

1. Create an isolated virtual network on the hypervisor.
2. Create **VM 2 (Server)** with two adapters: NAT (temporary) and Host-only. Install Ubuntu/Debian. Give it a static lab IP.
3. Create **VM 1 (Attacker)** with one Host-only adapter. Install or import Kali Linux. Give it a static lab IP.
4. Copy `waf_demo_manager.py` to VM 2.
5. Run the script and choose **1 Setup**.
6. Verify the isolation with the checks in section 6.7.
7. Before the demonstration, remove the NAT adapter from VM 2.

6.2 Create the isolated network (VirtualBox example)

The exact menu names change between VirtualBox versions; the idea is always the same: create one **Host-only** network (or use an **Internal Network**) that is not connected to your real LAN or the internet.

1. Open VirtualBox, open the network manager (Tools, then Network or Host Network Manager).
2. Create a Host-only network. A common default is `192.168.56.1/24`. Leave DHCP disabled because we use static addresses.
3. Other hypervisors: VMware - a custom host-only VMnet; Proxmox/KVM - a Linux bridge with **no** uplink interface attached.

> **WARNING:** Do not use **Bridged** or **NAT** for the lab network. Only the temporary preparation adapter on VM 2 may use NAT.

6.3 Create VM 2 (Server)

1. Create a VM with 2 vCPU, 4 GB RAM and a 20 GB disk. Install **Ubuntu Server** (or Debian). A minimal install is enough; enabling OpenSSH server is convenient for copying the script.
2. VM settings, Network: **Adapter 1 = NAT** (temporary, for downloads) and **Adapter 2 = Host-only** (the lab network).
3. After installation, give the Host-only interface a static address. Find the interface names first:

```
ip -br link
```

Expected result: a list such as `lo`, `enp0s3` (NAT) and `enp0s8` (Host-only). Names differ per system.

Create a netplan file (replace `enp0s8` with your Host-only interface name):

```
sudo nano /etc/netplan/60-lab.yaml

network:
  version: 2
  ethernets:
    enp0s8:
      dhcp4: false
      addresses: [192.168.56.20/24]
```

```
sudo chmod 600 /etc/netplan/60-lab.yaml
sudo netplan apply
ip -4 addr show enp0s8
```

Expected result: the output contains `inet 192.168.56.20/24`.

6.4 Create VM 1 (Attacker)

1. Import or install **Kali Linux** (2 vCPU, 2 GB RAM).
2. VM settings, Network: **one adapter = Host-only** (the same network as VM 2). No NAT.
3. Give it a static address in the same subnet, for example with the network settings of the desktop or with `nmcli`:

```
nmcli con show                                # find the connection name
sudo nmcli con mod "Wired connection 1" ipv4.method manual \
     ipv4.addresses 192.168.56.10/24
sudo nmcli con up "Wired connection 1"
```

```
ping -c 3 192.168.56.20
```

Expected result: three replies from 192.168.56.20. If you get no reply, check that both VMs use the same Host-only network and that the addresses are correct.

6.5 Copy the script to VM 2

Any method works. With the Host-only network and OpenSSH on VM 2, copy it from your host computer:

```
scp waf_demo_manager.py youruser@192.168.56.20:~/
```

Expected result: the file appears in the home directory of `youruser` on VM 2 (`ls -l ~/waf_demo_manager.py`).

Alternatives: a VirtualBox shared folder, a USB stick, or downloading it while the NAT adapter is connected.

6.6 Run the script for the first time

```
cd ~
python3 waf_demo_manager.py
```

Expected result: a short line says that root rights are needed and the script re-runs itself with `sudo` (enter your password). The banner and the main menu then appear. If you are already root, no sudo step happens.

Choose **1 Setup**. This is what you will be asked and what happens:

1. **Pre-flight warnings** appear only if something is wrong (little disk space, little memory, no internet).
2. **Security notice**: read it and type `I UNDERSTAND` to continue. This is asked only once.
3. **Installing Docker** (only if it is missing): spinner lines for each step. Allow a few minutes.
4. **Question: "Where will the attacker machine connect from?"** Choose `1` (another machine on the lab network) for the two-VM demo, or `2` for this machine only.
5. **Downloading images**: Juice Shop and the WAF. This is the slowest step.
6. **Starting containers**, then **waiting until the WAF answers** (Juice Shop can take a minute to start).
7. **Quick check**: two PASS/FAIL lines, then the address the attacker should use.

```
[ OK ] Downloaded bkimminich/juice-shop:latest
[ OK ] Downloaded owasp/modsecurity-crs:nginx
[ OK ] Starting Juice Shop + WAF
[ OK ] Waiting for the WAF to proxy Juice Shop (14s)
  PASS  Normal request reaches Juice Shop  (HTTP 200)
  PASS  XSS probe forwarded and logged  (HTTP 200)

  Attacker should target:
    http://192.168.56.20:8080/
  Juice Shop itself is NOT published - it can only be reached through the WAF.
```

Illustrative output. The quick check runs with protection OFF (the default), so the XSS probe is expected to be "forwarded and logged", not blocked.

> **TIP:** While VM 2 still has its NAT adapter, the script may list **two** addresses. Use the one on your lab subnet (for example 192.168.56.20), not the NAT address.

6.7 Verify the isolation

Run these checks before every demonstration. They prove that the shop can only be reached through the WAF.

From VM 1 (attacker):

```
curl -m 5 -i http://192.168.56.20:3000
```

Expected result: failure: `Connection refused` or `Connection timed out`. Juice Shop must NOT answer on port 3000.

```
curl -i http://192.168.56.20:8080/
```

Expected result: `HTTP/1.1 200 OK` followed by the HTML of the Juice Shop page, delivered through the WAF.

On VM 2 (server):

```
sudo docker ps --format 'table {{.Names}}\t{{.Ports}}'
```

Expected result: `wafdemo-waf` shows a published port like `0.0.0.0:8080->8080/tcp`; `wafdemo-juice-shop` shows **no** published port.

```
sudo docker network inspect wafdemo-net --format '{{range .Containers}}{{.Name}} {{end}}'
```

Expected result: exactly two names: `wafdemo-waf` and `wafdemo-juice-shop` (in any order).

You can also run **5 Tools, then 3 Show config & health check**. It prints "Everything looks healthy; Juice Shop is isolated (no published port)."

6.8 Before the demonstration: remove the NAT adapter

1. Shut down VM 2.
2. In the VM settings, disable or remove **Adapter 1 (NAT)** so only the Host-only adapter remains.
3. Start VM 2. Docker starts by itself and both containers come back automatically (their restart policy is `unless-stopped`).
4. Run the script and open **5 Tools, 3 Show config & health check** to confirm everything is healthy.
5. Run the isolation checks again.

> **NOTE:** The lab needs no internet while running. Removing NAT proves that to your audience and prevents accidental exposure.

7. Using the menu

The script has five choices. Each screen is cleared before the next one appears, so the audience always sees one thing at a time.

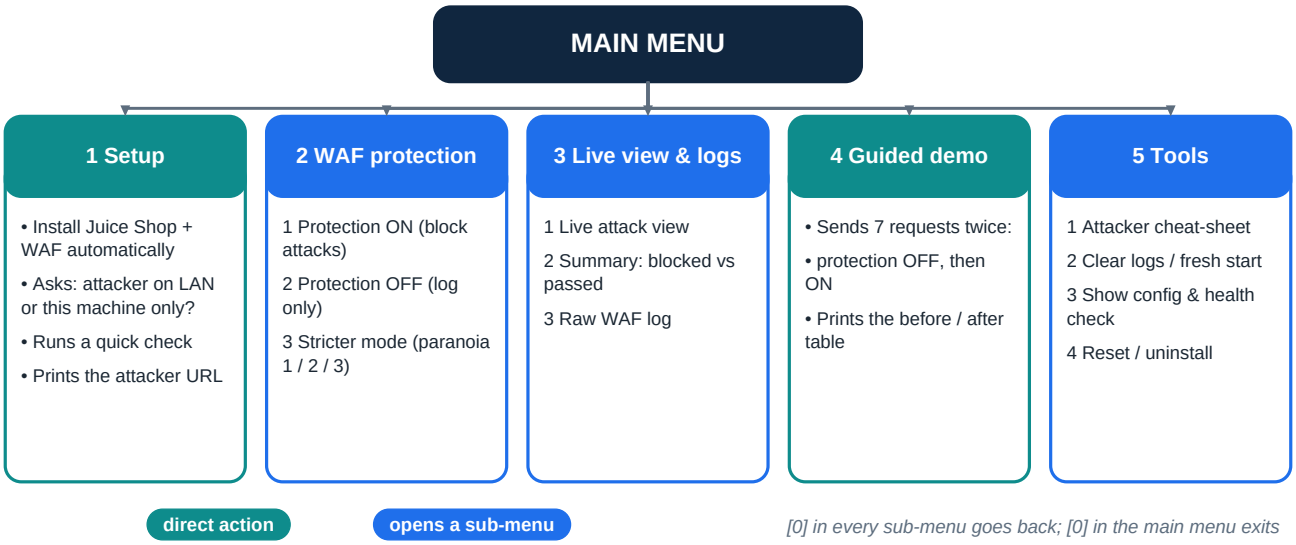


Figure 5. *The complete menu structure. Option 0 goes back (or exits from the main menu).*

The main menu looks like this (the script also shows a coloured banner above it, and a status line):

```
  ●  Juice Shop      ●  PROTECTION OFF      0.0.0.0:8080

                          MAIN MENU
  [1]  Setup              Install Juice Shop + WAF automatically
  [2]  WAF protection     Switch ON (block) / OFF (log only)
  [3]  Live view & logs   Attacker, attack type, result
  [4]  Guided demo        Automatic before/after attack test
  [5]  Tools              Cheat-sheet, clear logs, health check, reset
  [0]  Exit
```

The status line shows at a glance whether the lab is installed, whether protection is ON or OFF, and the address and port the WAF listens on.

7.1 Option 1: Setup

What it does	Installs and starts everything, then proves it works.
What you will see	Progress lines (spinner for long steps), a PASS/FAIL quick check, and the address the attacker should use.
When to use	The first time; after a reset; or to change who can connect (LAN or this machine only).
If already installed	It says so and asks whether you want to change the access (LAN / this machine only). Nothing is reinstalled.

Setup is safe to run again at any time. It also removes leftover containers from earlier versions of the tool.

7.2 Option 2: WAF protection

Choice	Effect	What you will see
1 Protection ON (block attacks)	ModSecurity rule engine = `On`. Attacks receive **403** and never reach Juice Shop.	Container is recreated, the script waits until the WAF answers, then prints that protection is ON.
2 Protection OFF (log only)	Rule engine = `DetectionOnly`. Attacks pass but are logged.	Same, ending with "PROTECTION OFF: attacks reach Juice Shop, but the WAF still logs them."
3 Stricter mode (paranoia 1/2/3)	Raises the strictness of the rules. Optional.	A prompt for the level (default 1). Higher levels may block legitimate requests.

> **NOTE:** Switching takes several seconds because the WAF container is recreated (section 8.9). Juice Shop keeps running. Earlier log events are saved first, so they are not lost.

7.3 Option 3: Live view & logs

Choice	What it shows
1 Live attack view	A live feed (stop with Ctrl+C). One line per request that triggers a WAF rule: time, attacker IP, attack type in plain words, result and the request. Normal browsing is not listed, because ModSecurity only logs requests that match rules.
2 Summary: blocked vs passed	Totals seen by the WAF since its last restart (allowed vs 403); a table of attacks **by type** with blocked and passed counts; the 10 most recent attacks; the 5 most frequent CRS rules. Includes events saved before earlier restarts.
3 Raw WAF log	The last 30 lines exactly as the WAF container wrote them. Use it for troubleshooting.

```
TIME     ATTACKER IP     ATTACK TYPE          RESULT               REQUEST
-------------------------------------------------------------------------------
10:02:11 192.168.56.10   SQL injection        PASSED - logged only  GET   /rest/products/
search?q=app...
10:02:19 192.168.56.10   Cross-site scripting  PASSED - logged only  GET   /rest/products/
search?q=%3C...
10:03:02 192.168.56.10   SQL injection        BLOCKED (403)         GET   /rest/products/
search?q=app...
10:03:09 192.168.56.10   Scanner / attack tool BLOCKED (403)        GET   /
```

Illustrative. Meaning of the result column:

Result shown	Meaning
BLOCKED (403)	The WAF stopped the request and answered 403. It never reached Juice Shop.
PASSED - logged only	The attack score reached the threshold, but protection is OFF, so the request was forwarded. The WAF logged that it *would have* been blocked.
PASSED - flagged	At least one rule matched but the total score stayed below the threshold; the request was allowed.

7.4 Option 4: Guided demo

The script sends the same seven requests twice, first with protection OFF and then with protection ON, against the lab WAF on the server itself, and prints a comparison table. It switches the mode for you and restores the previous mode afterwards. It takes roughly a minute, mostly spent waiting for the WAF to restart twice.

```
Attack                          | Protection OFF          | Protection ON
-------------------------------------------------------------------------------
SQL injection (login bypass)    | passed through (200)    | BLOCKED (403)
SQL injection (search UNION)    | passed through (200)    | BLOCKED (403)
Cross-site scripting            | passed through (200)    | BLOCKED (403)
Path traversal                  | passed through (200)    | BLOCKED (403)
Command injection               | passed through (200)    | BLOCKED (403)
Automated scanner (sqlmap)      | passed through (200)    | BLOCKED (403)
Business logic: basket tampering| passed through (200)    | STILL PASSES (200)
Result: with protection ON the WAF blocked 6 of 6 attack-pattern requests.
```

Illustrative. The HTTP codes in the "passed through" cells depend on how Juice Shop answers (for example 200 or 500). Any code other than 403 means the request reached the application.

> **NOTE:** Each run creates a temporary demo account inside Juice Shop for the business-logic request. It disappears when the lab is reset.

7.5 Option 5: Tools

Choice	What it does	Details
1 Attacker cheat-sheet	Prints ready-to-copy commands for VM 1 with the real server address filled in.	Normal request, two SQL injections, XSS, path traversal, command injection, `nikto`, `sqlmap` (section 10.4).
2 Clear logs / fresh start	Gives the audience a clean slate.	Deletes the saved event journal and recreates the WAF container, which empties its logs.
3 Show config & health check	Shows settings, containers and the WAF's effective environment; checks health.	Verifies Docker, both containers, isolation (shop publishes no port) and that the WAF answers. If a problem is found it offers **automatic repair**.
4 Reset / uninstall	Removes the lab containers and network.	Asks for confirmation (default No). Downloaded images are kept, so Setup is fast the next time.

8. How the script works internally

This section is for implementers. It follows the structure of `waf_demo_manager.py` from the outside in.

8.1 Design principles

- **One file, standard library only.** Nothing to install except Docker; the script can be copied anywhere.
- **Everything automated, with fallbacks.** Each step that can fail has an alternative (section 8.4 and 8.5).
- **Safe by default.** The WAF binds to `127.0.0.1` until you choose LAN access; the shop never publishes a port; a notice must be acknowledged once.
- **Idempotent.** Running Setup or repair again never harms an installed lab. At every start the script compares its saved state with the containers that really exist (`reconcile`).
- **Never crash the menu.** Unexpected errors are caught, shown briefly and written to `error.log`.

8.2 Code map

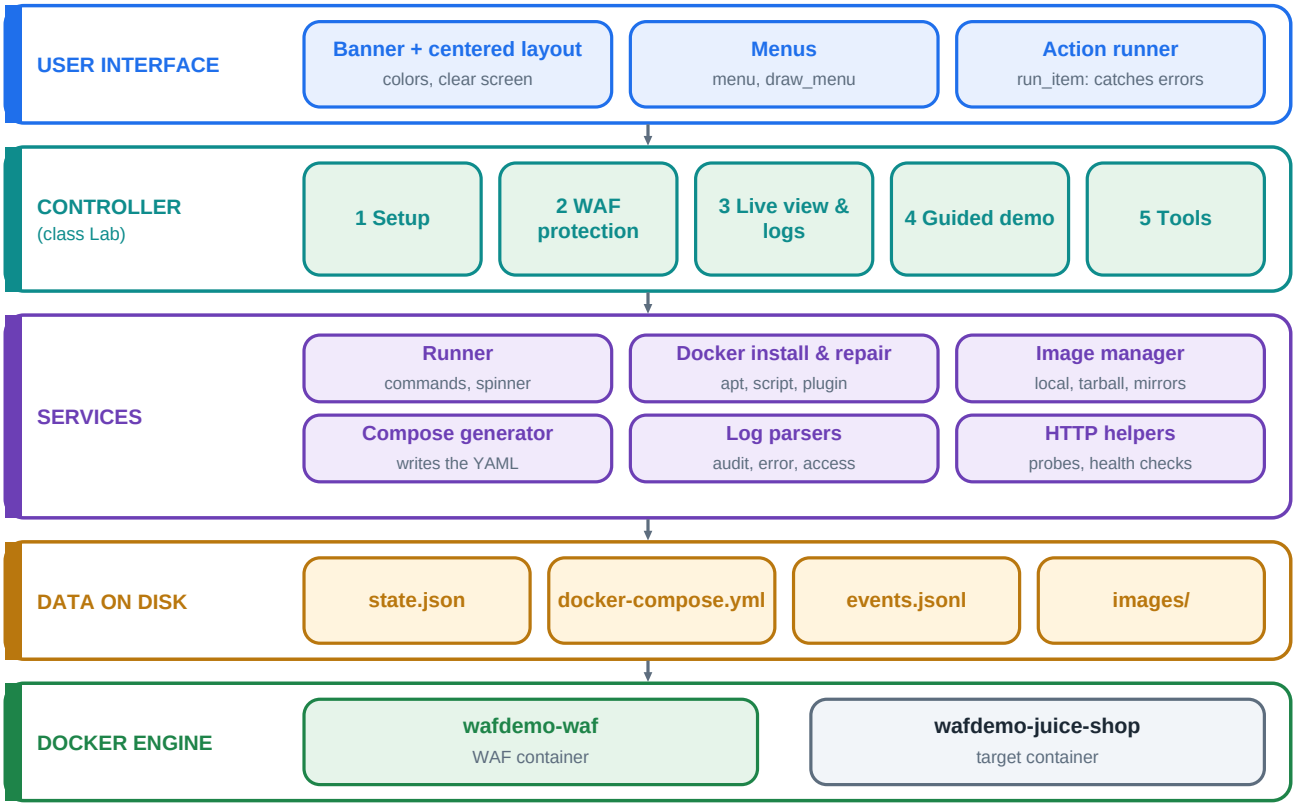


Figure 6. The script in layers. The menu calls the controller (class Lab); the controller uses small services and talks to Docker.

Part of the file	Main names	Purpose
Terminal helpers	`_c`, `out`, `box`, `center_line`, `banner_lines`, `clear`	Colours, centred layout, banner, clearing the screen, fallback for terminals without UTF-8 or colour.
State	`class State`, `DEFAULT_STATE`	Reads and writes `state.json` atomically.
Images and compose	`IMAGES`, `COMPOSE_TEMPLATE`	Image names with fallbacks; the text of `docker-compose.yml`.
Log parsing	`parse_audit_line`, `events_from_errorlog`, `parse_access`, `LiveParser`, `attack_type`, `verdict_for`	Turn WAF log lines into one event per request and name the attack.

Part of the file	Main names	Purpose
Process helpers	`class Runner` (`run`, `task`, `stream`), `http_request`, `port_free`, `lan_ips`	Run commands with a spinner, stream output, probe HTTP, find free ports and the LAN address.
Demo requests	`DEMO_ATTACKS`, `business_logic_probe`	The seven requests of the guided demo.
Controller	`class Lab`	All actions: Docker install, images, setup, protection, logs, demo, tools, repair.
Menus	`menu`, `draw_menu`, `run_item`, `build_menus`	Draw each screen, read the choice, run the action, pause.
Entry point	`parse_args`, `elevate`, `main`	Command-line options, sudo re-launch, choose menu or one-shot mode.

8.3 Startup and sudo elevation

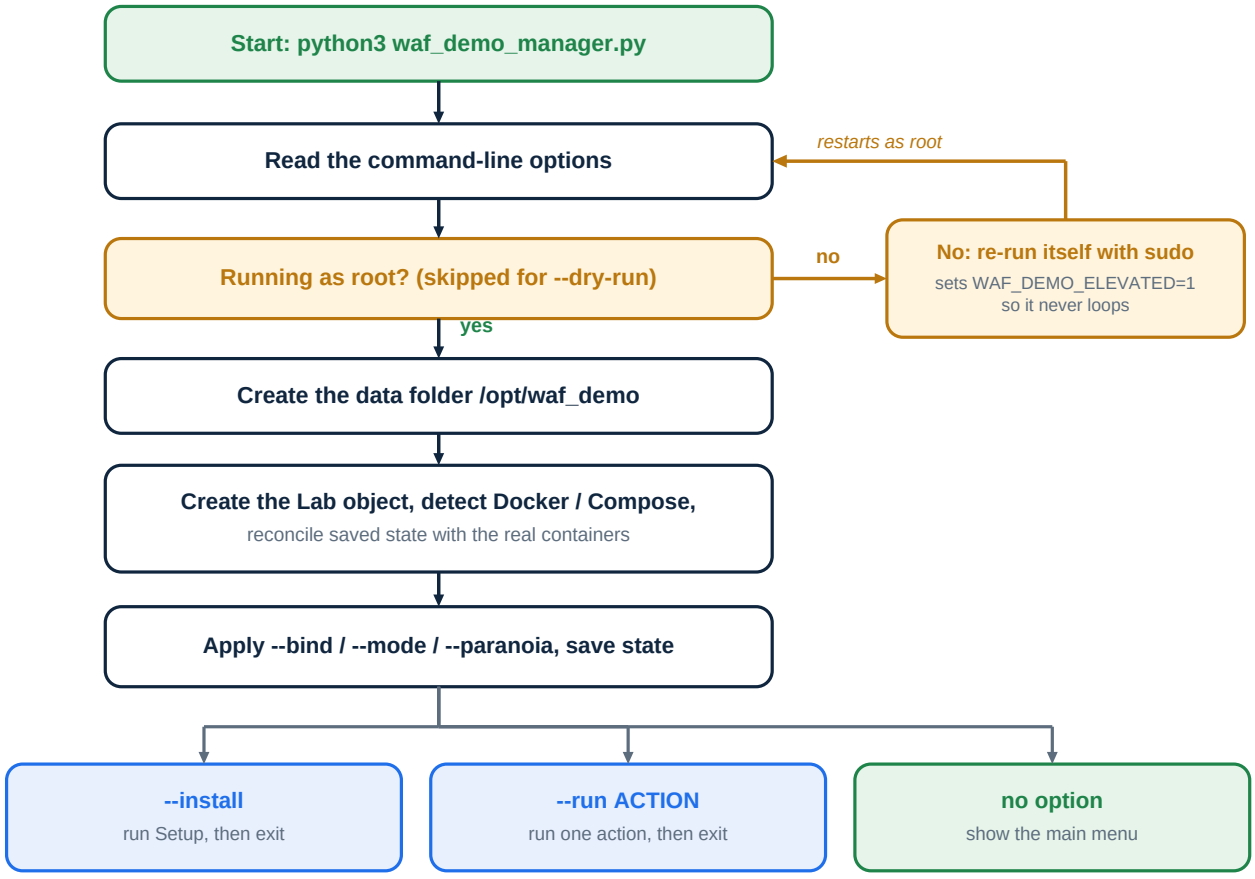


Figure 7. *What happens between typing the command and seeing the menu.*

- If the script is not root (and not in `--dry-run`), it replaces itself with `sudo env WAF_DEMO_ELEVATED=1 python3 waf_demo_manager.py ...`. The marker variable prevents an endless loop.
- The data folder is `/opt/waf_demo` when root, otherwise `~/.waf_demo`; the variable `WAF_DEMO_HOME` overrides it.
- `reconcile()` asks Docker which `wafdemo-*` containers exist and corrects the saved `installed` flag.

8.4 Automatic Docker installation and repair

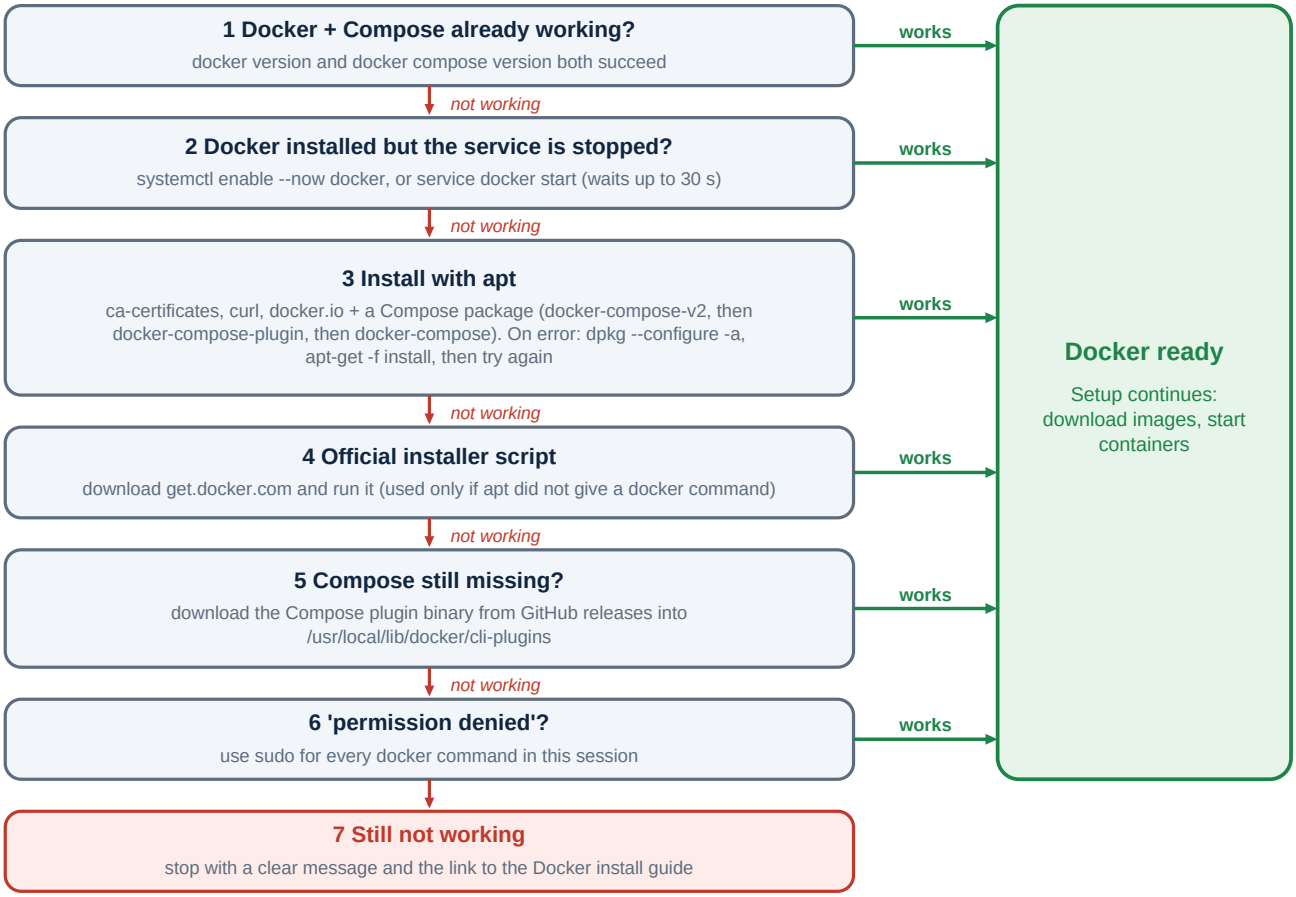


Figure 8. The chain of alternatives used by `ensure_docker`. The script moves to the next step only when the previous one did not work.

- `apt-get` runs with a lock timeout of 300 seconds, so it waits politely if Ubuntu's background updater holds the package lock.
- Compose is detected as `docker compose` (v2 plugin) first, then the old `docker-compose` command.
- If Docker is installed but the user is not allowed to talk to it, the script prefixes docker commands with `sudo` for the rest of the session. It never adds you to the `docker` group.

8.5 Getting the images

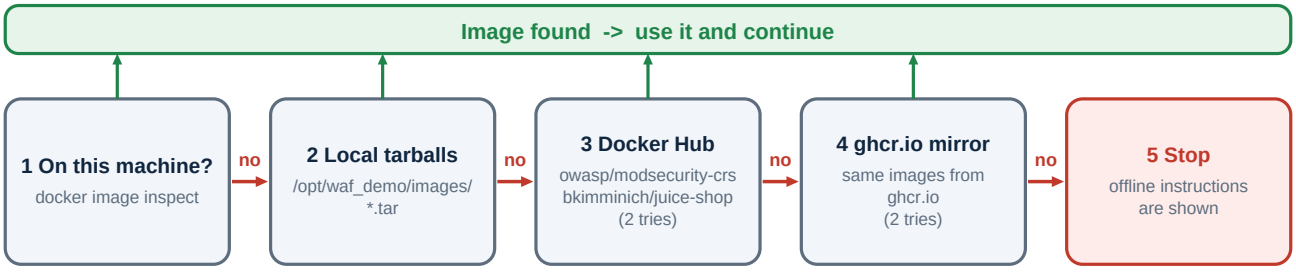


Figure 9. `ensure_image`: where the script looks for each of the two images.

Image	Primary source	Mirror (second source)
WAF	`owasp/modsecurity-crs:nginx`	`ghcr.io/coreruleset/modsecurity-crs:nginx`
Juice Shop	`bkimminich/juice-shop:latest`	`ghcr.io/juice-shop/juice-shop:latest`

> **TO BE VERIFIED:** The mirror names are a fallback; confirm that they exist at the time you use them. The `latest` tag is not pinned, so a future Juice Shop version may behave differently.

The chosen image is saved in `state.json`, so later runs reuse the same one.

8.6 The generated compose file

The script writes `/opt/waf_demo/docker-compose.yml` from a template and passes the current settings as environment variables when it calls Compose. The complete file is in Appendix B. The lines that matter most:

Line	Why it matters
`juice-shop` has **no** `ports:` entry	Makes the shop unreachable except through the WAF.
`waf` publishes `${LAB_BIND}:${LAB_WAF_PORT}:8080`	The only open door. `LAB_BIND` is `127.0.0.1` (this machine) or `0.0.0.0` (LAN).
`BACKEND=http://juice-shop:3000`	Tells the reverse proxy where to forward allowed requests, using the Docker service name.
`MODSEC_RULE_ENGINE=${LAB_RULE_ENGINE}`	`DetectionOnly` (protection OFF) or `On` (protection ON).
`PARANOIA=${LAB_PARANOIA}`	Rule strictness, 1 to 3.
`ANOMALY_INBOUND=5`, `ANOMALY_OUTBOUND=4`	Score thresholds (the CRS defaults).
`MODSEC_AUDIT_LOG=/dev/stdout`, `..._FORMAT=JSON`	Sends the audit log to the container output so `docker logs` can read it.
`restart: unless-stopped`	Containers come back after a reboot.
`security_opt: no-new-privileges`	A small hardening measure for both containers.

8.7 State and files on disk

`state.json` remembers the choices between runs:

Field	Meaning	Default
`installed`	Both containers exist	false
`bind`, `bind_chosen`	Address the WAF listens on, and whether you have been asked	`127.0.0.1`, false
`waf_port`	Published port (changes automatically if busy)	8080
`paranoia`	Strictness level	1
`rule_engine`	`DetectionOnly` = protection OFF, `On` = protection ON	`DetectionOnly`
`acknowledged`	The security notice was accepted	false
`images`	The image names actually used	(empty)

8.8 What Setup does, step by step

1. `prepare()`: pre-flight (silent unless problems), security notice, make sure Docker and Compose work.
2. `legacy_cleanup()`: remove containers/network left by older versions of the tool.
3. If both containers are already running: report it, offer to change the access, show the URL, stop.
4. Ask who will connect (LAN or this machine only) if not asked yet.
5. `ensure_image()` for Juice Shop and for the WAF.
6. `pick_port()`: if the chosen port is busy, use the next free one and tell the user.
7. Write `docker-compose.yml`; run `docker compose up -d`. On failure, retry once with `--force-recreate`.

8. If the firewall `ufw` is active and the bind is not local, open the WAF port.
9. `wait_ready()`: poll the WAF address until it answers with a code below 500 (up to 200 s; after 60 s of silence `heal_stack()` runs).
10. `quick_check()`: a normal request and an XSS probe, judged according to the current mode.
11. Show the attacker URL(s).

8.9 Switching protection ON or OFF

ModSecurity reads its settings from environment variables **when the container starts**. To change the mode the script therefore (1) stores the new value, (2) saves current WAF events to the host journal, (3) recreates **only the WAF container** (`docker compose up -d --force-recreate waf`), and (4) waits until the WAF answers again. Juice Shop is untouched, so it stays running. The same procedure applies to the paranoia level.

8.10 Port selection and firewall

- `pick_port()` tries to bind the chosen address and port; if that fails it tries the next 29 ports and saves the first free one. It skips the check when the WAF container is already running, because that container is the one holding the port.
- `open_firewall()` adds an `allow <port>/tcp` rule only when `ufw` is installed and active and the bind is not local. Docker's own rules normally bypass `ufw` anyway; this is a safety net.

8.11 The log pipeline

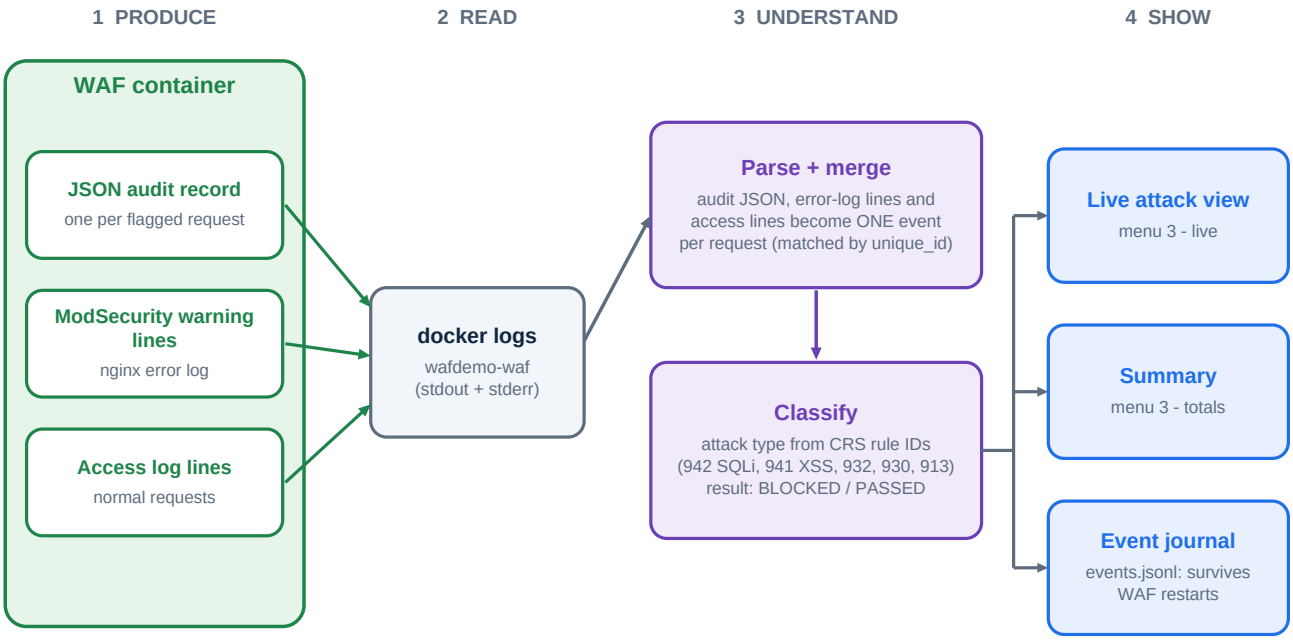


Figure 10. *From the WAF container to the screen. Three kinds of log lines are merged into one event per request.*

The WAF container writes everything to its output, which `docker logs` returns:

- a **JSON audit record** for each request that triggered rules (attacker IP, request, response code, the rule IDs and messages);
- **ModSecurity lines** in the nginx error log (one per matched rule, plus a summary line when the threshold is reached);
- the ordinary **access log** (every request, used for the allowed/blocked totals).

`LiveParser` and `events_from_lines` match these lines through ModSecurity's `unique_id`, so a request is shown **once**, even if it appears in two logs. If audit records are missing, the error-log lines alone still give a correct

result.

The attack type comes from the first three digits of the CRS rule ID (full table in Appendix D). The result label comes from this rule:

Condition	Result shown
Response code is 403	BLOCKED (403)
Anomaly threshold exceeded but not blocked (protection OFF)	PASSED - logged only
Rules matched but score below the threshold	PASSED - flagged
No rule matched	ALLOWED (not listed in the live view)

Event journal. Because the WAF container is recreated when the mode changes, its logs would vanish. Before each recreation the script appends its events to `/opt/waf_demo/events.jsonl`. The summary combines the journal with the current logs, so the numbers survive mode switches. "Clear logs / fresh start" deletes the journal.

8.12 The guided demo, in detail

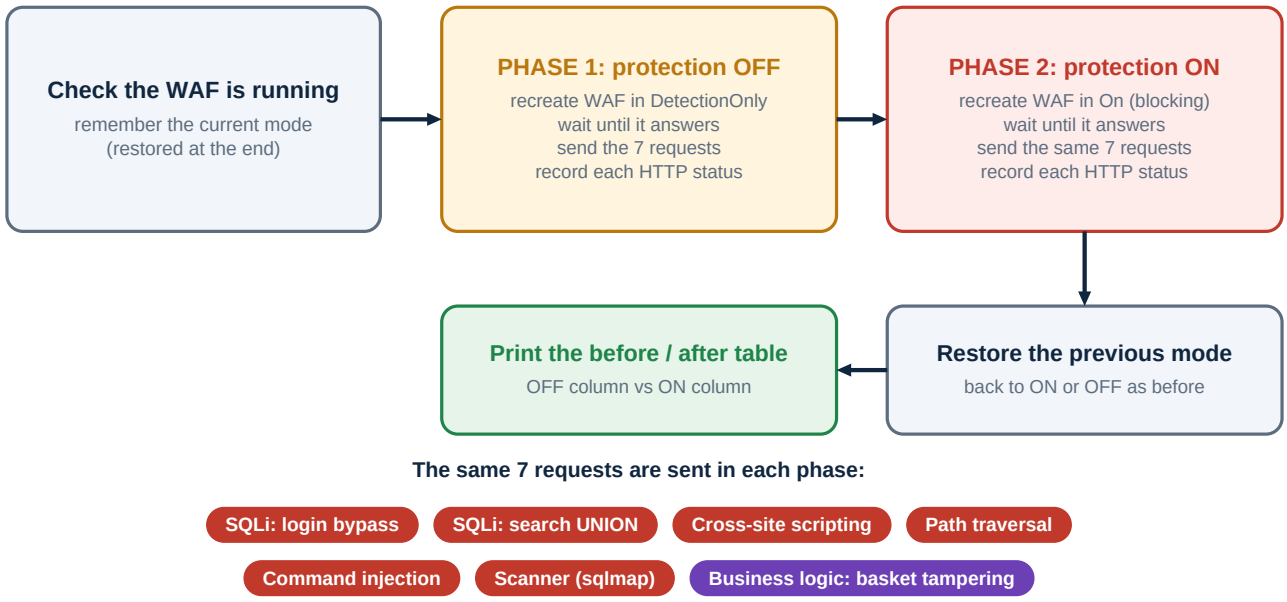


Figure 11. Flow of the guided demo.

#	Request	What is sent	Rule family expected
1	SQL injection (login bypass)	POST `/rest/user/login`, JSON `{"email":"' OR 1=1--","password":"x"}`	942xxx
2	SQL injection (search UNION)	GET `/rest/products/search?q=apple'))%20UNION%20SELECT%20*%20FROM%20Users--`	942xxx
3	Cross-site scripting	GET `/rest/products/search?q=<script>alert(1)</script>` (URL-encoded)	941xxx
4	Path traversal	GET `/rest/products/search?q=../../../../etc/passwd` (URL-encoded)	930xxx
5	Command injection	GET `/rest/products/search?q=;cat /etc/passwd` (URL-encoded)	932xxx
6	Automated scanner	GET `/` with User-Agent `sqlmap/1.7.2`	913xxx

#	Request	What is sent	Rule family expected
7	Business logic	Register a temporary user, log in, then POST `/api/BasketItems` with two `BasketId` keys (own and another user's basket)	none

Request 7 contains no attack pattern. It is an authenticated, well-formed request that abuses a **broken access control** weakness, so it reaches the application in both phases. A request counts as "blocked" in the table when the answer is exactly HTTP 403; any other code is "passed through".

8.13 Self-repair behaviour

Situation	What the script does
Docker is installed but stopped	Starts the service (`systemctl` or `service`), waits up to 30 seconds.
Docker or Compose is missing	Installs it using the fallback chain (section 8.4).
Image download fails	Retries, then tries the mirror, then local tarballs; otherwise explains the offline procedure.
Port already in use	Chooses the next free port.
WAF does not answer for 60 seconds	`heal_stack()`: starts all containers, waits 8 s, recreates the WAF (nginx may have kept an outdated address for the shop).
Containers missing or stopped (health check)	`repair()`: re-checks images and recreates both containers, then waits for the WAF.
Leftovers from older versions	`legacy_cleanup()` removes them before installing.
Unexpected Python error in a menu action	Shown briefly, full details written to `/opt/waf_demo/error.log`, menu continues.

8.14 User interface behaviour

- Every menu and every action starts with a cleared screen, a banner and the status line.
- Text is centred in a column up to 110 characters wide; menus are also centred vertically.
- Colours are used only when the output is a terminal; set `NO_COLOR=1` to disable them. If the terminal is not UTF-8, plain ASCII symbols replace box and arrow characters.
- Long commands show a spinner with elapsed seconds and print the last lines of output only if they fail.
- `--dry-run` prints the commands that would run without executing the changing ones.

9. Running the demonstration

A ready-to-use presenter script: about 40 minutes, seven short acts. Do the pre-demo checklist (Appendix F) first.

9.1 Room and screen layout

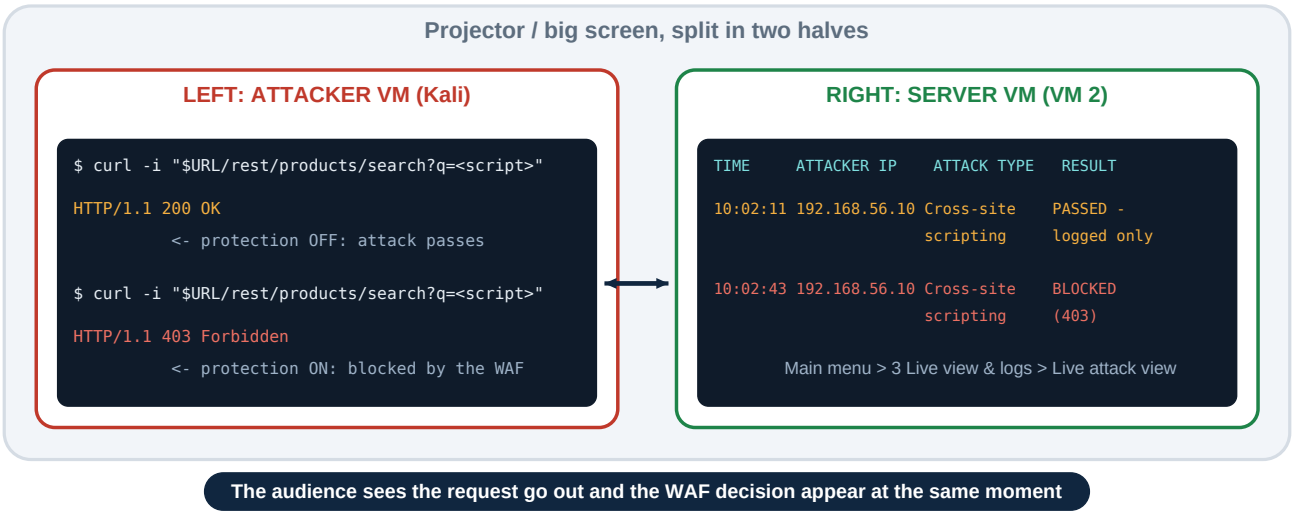


Figure 12. Show both machines on one big screen: the attack on the left, the WAF's decision on the right, at the same moment.

On VM 2 keep the menu open at **3 Live view & logs, 1 Live attack view** during the attack acts. Open a second terminal on VM 1 with the address variable ready:

```
URL=http://192.168.56.20:8080
```

Expected result: no output. Use the address printed by Setup.

9.2 Timeline and talk track

Act	Min	You do	You say	Audience sees
0 Introduce	3	Show the topology diagram (Figure 2).	"Two machines. The attacker, and a web shop with a guard in front. The shop has no door of its own; the guard is the only way in."	The diagram.
1 Normal use	3	VM 1 browser: open the shop at the WAF address, add an item to the basket. VM 2: Live attack view is empty.	"A WAF is invisible to honest users. Nothing is listed because nothing suspicious happened."	The shop works normally.
2 Guard only watching	7	VM 2: **2, 2 Protection OFF**. VM 1: run the cheat-sheet attacks (SQL injection login, search, XSS, traversal, command injection).	"The attacks work. The WAF noticed every one and wrote it down, but it was told only to watch. Many companies start like this to tune the rules safely."	VM 1: HTTP 200 answers (the login bypass returns a token). VM 2: lines `PASSED - logged only` with the attack type.
3 Guard blocking	7	VM 2: **2, 1 Protection ON**. VM 1: repeat the same commands. VM 2: open **3, 2 Summary**.	"Same attacks, same rules, one switch. The shop never saw them."	VM 1: `HTTP/1.1 403 Forbidden`. VM 2: `BLOCKED (403)` lines, then the by-type table.
4 Scanners	4	VM 1: `nikto -h $URL` or the sqlmap command. Stop after a few seconds with Ctrl+C.	"Automatic tools are noisy; the rules recognise their signature."	A stream of 403 answers; the live view fills with `Scanner / attack tool`.

Act	Min	You do	You say	Audience sees
5 Guided demo	5	VM 2: **4 Guided demo**. Wait about a minute.	"The script repeats it all, once with protection OFF and once ON, and compares."	The before/after table.
6 What a WAF cannot do	5	Point to the last row of the table.	"This request is perfectly normal on its face. The flaw is in the shop's logic. The WAF cannot fix that; the developers must."	`STILL PASSES` in the ON column.
7 Optional: stricter mode	3	VM 2: **2, 3 Stricter mode**, level 3. Try a search containing unusual characters on VM 1. Then set level 1 again.	"Stricter rules catch more but start blocking honest users. Operators tune with exclusions."	Possibly a legitimate request being blocked (not guaranteed).
8 Wrap-up	3	VM 2: **5, 2 Clear logs** for the next session.	"A WAF is a fast, valuable layer, not a complete defence."	

TO BE VERIFIED: Act 7 depends on the CRS version in the image; whether a given harmless search is blocked at level 2 or 3 must be tested beforehand. Prepare the example during rehearsal.

9.3 Presenter tips

- Rehearse once with the whole chain, including a mode switch, so you know how long the WAF takes to come back (usually a few seconds).
- Use `curl` for the cross-site-scripting demonstration. A browser may still run script that is handled entirely by the page itself (client-side), even when the WAF blocks the HTTP request that carried it, which would confuse the message.
- The login SQL injection also works in the browser: type `' OR 1=1--` as the e-mail and any password in the Juice Shop login form. With protection ON the browser shows a failed login and the developer tools show the 403.
- Keep the Guided demo as a backup: if the attacker VM misbehaves, it produces the same evidence from VM 2 alone.

9.4 If something goes wrong during the demo

Symptom	Quick action
Live view stays empty after attacks	Open **3, 2 Summary** (it also reads saved events) or **3, 3 Raw WAF log**. Make sure the attacks use the WAF port, not 3000.
Switching protection takes long	Wait; the WAF is restarting. The script waits up to 90 s and repairs itself after 60 s without an answer.
Guided demo shows "no response"	The WAF was still restarting. Run **5, 3 Show config & health check**, accept the repair, then run the demo again.
The attacker cannot connect	Check the IP address, the Host-only network, and that the WAF was installed with LAN access (Setup asks again if you run it).

10. Command reference

10.1 Starting the script

```
python3 waf_demo_manager.py
```

Expected result: the banner and main menu. The script asks for your sudo password if you are not root.

```
python3 waf_demo_manager.py --version
```

Expected result: `1.0 (www.tca.ac.tz)`

10.2 Command-line options

Option	Meaning
`--install`	Run **Setup** without the menu (installs Juice Shop and the WAF), then exit. Exit code 0 on success, 1 on failure.
`--run ACTION`	Run one action and exit (list in section 10.3).
`--mode detect|block`	Set the initial mode: `detect` = protection OFF, `block` = protection ON.
`--paranoia 1|2|3`	Set the paranoia level.
`--bind ADDRESS`	Address the WAF listens on: `127.0.0.1` (this machine) or `0.0.0.0` (LAN). Counts as having answered the Setup question.
`--yes`, `-y`	Answer every question automatically with its default (no prompts).
`--accept-risk`	Accept the vulnerable-application notice without typing `I UNDERSTAND`.
`--dry-run`	Show the commands and files that would be changed without changing anything; skips the sudo re-launch.
`--version`	Print the version and exit.

Example: a completely unattended installation for a LAN demo with protection ON:

```
python3 waf_demo_manager.py --install --yes --accept-risk --bind 0.0.0.0 --mode block
```

Expected result: Setup runs without questions and ends by printing the attacker URL.

Environment variables used by the script itself:

Variable	Effect
`WAF_DEMO_HOME`	Use another data folder instead of `/opt/waf_demo` (or `~/.waf_demo` for non-root).
`NO_COLOR`	Set to any value to switch colours off.
`WAF_DEMO_ELEVATED`	Internal marker set by the script after it re-launches itself with sudo. Do not set it yourself.

10.3 The --run actions

Action	Same as menu	Does
`setup`	1	Install and start the lab
`protection_on`	2, 1	Switch to blocking
`protection_off`	2, 2	Switch to log-only

Action	Same as menu	Does
`live`	3, 1	Live attack view (Ctrl+C to stop)
`summary`	3, 2	Blocked vs passed summary
`raw`	3, 3	Last 30 raw WAF log lines
`demo`	4	Guided demo
`cheatsheet`	5, 1	Print attacker commands
`clear_logs`	5, 2	Fresh start for the logs
`config`	5, 3	Show config and health check
`repair`	(health check)	Recreate both containers and wait until the WAF answers
`reset`	5, 4	Remove the lab containers and network

10.4 Attacker commands and expected results

Run these on **VM 1**. Replace the address with the one printed by Setup. The same commands are printed, with your real address, by **5 Tools, 1 Attacker cheat-sheet**.

```
URL=http://192.168.56.20:8080
```

Normal request

```
curl -i "$URL/"
```

Expected result: OFF and ON: `HTTP/1.1 200 OK` and the HTML of the shop. Not listed in the WAF log.

SQL injection - search

```
curl -i "$URL/rest/products/search?q=apple%27))%20UNION%20SELECT%20*%20FROM%20Users--"
```

Expected result: OFF: the request reaches Juice Shop; the answer may be a normal result or an application error (either proves it got through). WAF log: `SQL injection`, `PASSED - logged only`. ON: `HTTP/1.1 403 Forbidden`, log shows `BLOCKED (403)`.

SQL injection - login bypass

```
curl -i -X POST $URL/rest/user/login -H "Content-Type: application/json" \
     -d "{\"email\":\"' OR 1=1--\",\"password\":\"x\"}"
```

Expected result: OFF: Juice Shop logs the attacker in (a JSON answer with an authentication token). ON: `HTTP/1.1 403 Forbidden`.

Cross-site scripting

```
curl -i "$URL/rest/products/search?q=%3Cscript%3Ealert(1)%3C%2Fscript%3E"
```

Expected result: OFF: `HTTP/1.1 200 OK`, logged as `Cross-site scripting`. ON: `HTTP/1.1 403 Forbidden`.

Path traversal

```
curl -i "$URL/rest/products/search?q=..%2F..%2F..%2F..%2Fetc%2Fpasswd"
```

Expected result: OFF: reaches Juice Shop (200); logged as `Path traversal`. ON: `HTTP/1.1 403 Forbidden`. The request is not harmful to the server; the app simply searches for that text.

Command injection

```
curl -i "$URL/rest/products/search?q=%3Bcat%20%2Fetc%2Fpasswd"
```

Expected result: OFF: reaches Juice Shop (200); logged as `Command injection`. ON: `HTTP/1.1 403 Forbidden`. Juice Shop is a Node.js app, so the command is not really executed.

Scanner - nikto

```
nikto -h $URL
```

Expected result: OFF: the scan proceeds and fills the WAF log. ON: most probes receive 403 within the first few requests. Stop it with Ctrl+C when you have made the point.

Scanner - sqlmap

```
sqlmap -u "$URL/rest/products/search?q=test" --batch
```

Expected result: OFF: sqlmap starts testing parameters; the WAF logs scanner and injection patterns. ON: sqlmap is blocked early (it announces `403` or connection problems).

> **TO BE VERIFIED:** The exact answers of Juice Shop (200, 500 or a JSON error) and how many probes `nikto` and `sqlmap` get through before being blocked depend on the Juice Shop and CRS versions in your images. Record your own results during rehearsal.

10.5 Reading the results

Where	Text	Meaning
Live view / summary	`BLOCKED (403)`	The WAF answered 403; the shop never saw the request.
Live view / summary	`PASSED - logged only`	Threshold reached, protection OFF: forwarded and logged.
Live view / summary	`PASSED - flagged`	A rule matched, but below the threshold: allowed.
Guided demo table	`passed through (N)`	Any HTTP code except 403: the request reached the application.
Guided demo table	`BLOCKED (403)`	Protection ON: stopped by the WAF.
Guided demo table	`STILL PASSES (N)`	Business-logic row with protection ON: not blocked, as expected.
Guided demo table	`NOT blocked (N)`	An attack-pattern row was not blocked with protection ON: investigate (paranoia, rule set, mode).

11. Troubleshooting

Find your symptom in the table. Always try **5 Tools, 3 Show config & health check** first: it detects most problems and offers automatic repair.

Symptom	Likely cause	What to do
Docker installation fails	No internet on VM 2, or the package manager is locked by another process	Connect the NAT adapter, wait a few minutes for background updates to finish, then run **1 Setup** again. The script retries with a 5-minute lock wait and repairs interrupted package installs automatically.
"No internet to the image registries" warning / image download fails	VM 2 has no route to Docker Hub or ghcr.io	Reconnect the NAT adapter, or use the offline method below.
"Port 8080 is already in use - using 8081"	Another program uses 8080	Nothing to fix: use the address printed by Setup. To see who uses it: `sudo ss -ltnp | grep 8080`.
WAF does not answer or returns 502 / Bad Gateway	Juice Shop is still starting, or nginx kept an old address	Wait up to a minute. After 60 s the script repairs itself. Manually: **5, 3** and accept the repair.
Live view or summary is empty	No rule-triggering request has arrived; attacks were sent to the wrong port; WAF was cleared	Send an attack (cheat-sheet). Attack the WAF port, not 3000. Open **3, 3 Raw WAF log** to see what the WAF writes. If the raw log shows ModSecurity lines but the views stay empty, record the lines and report them.
Attacker cannot connect to the WAF	WAF installed for "this machine only"; wrong IP; VMs on different networks; firewall	Run **1 Setup** and answer "yes" to change the access to LAN. Use the Host-only address, not the NAT address. `ping` between the VMs. If `ufw` is active the script opens the port for LAN access.
Cheat-sheet shows the wrong IP	VM 2 has several network adapters; the script uses the first address found	Replace the address by the lab address (for example 192.168.56.20), or remove the NAT adapter.
Normal pages return 403 at paranoia 2 or 3	False positives of the stricter rules	Expected behaviour. Set paranoia back to 1 (**2, 3**).
Juice Shop is slow to start	First start needs time (and low memory slows it)	Setup waits up to 200 s. Give VM 2 at least 4 GB RAM.
"Root rights are needed"	No `sudo` and not root	Run the script as root or install `sudo`.
Strange symbols or no colours	Terminal is not UTF-8 or has no colour support	Use a UTF-8 terminal (for example `export LANG=C.UTF-8`). The script falls back to plain ASCII; `NO_COLOR=1` removes colours.
Guided demo says "no response"	WAF was restarting	Wait, run the health check, then run the demo again.
Setup says everything is already installed	Normal when the lab is running	Use **5, 4 Reset / uninstall**, then Setup, for a clean reinstall.
A menu action prints "Unexpected error"	A bug or an unusual environment	Read `/opt/waf_demo/error.log` and report its content with the step that caused it.

11.1 Installing the images offline

On any computer that has Docker **and** internet, download and save the two images:

```
docker pull bkimminich/juice-shop:latest
docker pull owasp/modsecurity-crs:nginx
docker save -o juice-shop.tar bkimminich/juice-shop:latest
docker save -o modsecurity-crs.tar owasp/modsecurity-crs:nginx
```

Copy the two `.tar` files to VM 2 and put them where the script looks for them:

```
sudo mkdir -p /opt/waf_demo/images
sudo cp juice-shop.tar modsecurity-crs.tar /opt/waf_demo/images/
```

Expected result: both files are listed by `ls /opt/waf_demo/images`. Run **1 Setup**: it prints that it is loading the local images and does not need the internet (Docker itself must already be installed).

11.2 Collecting information for a bug report

```
sudo docker ps -a --filter name=wafdemo-
sudo docker logs --tail 50 wafdemo-waf
sudo docker logs --tail 50 wafdemo-juice-shop
sudo cat /opt/waf_demo/state.json
sudo cat /opt/waf_demo/error.log
```

Expected result: the container list, the last log lines of both containers, the saved settings and (if present) the last error.

12. Security and ethics

> **WARNING:** This project deliberately runs a vulnerable application and uses attack tools. Use it **only on systems and networks that you own or are explicitly authorised to test**. Attacking systems without permission is illegal in most countries.

12.1 Isolation rules

- Both VMs must be on a **Host-only or Internal** network. Never **Bridged**, never **NAT** during the demonstration.
- Remove the temporary NAT adapter from VM 2 after preparation (section 6.8).
- Choose **LAN access** only on the isolated lab network. If you choose "this machine only", the WAF listens on `127.0.0.1` and nothing outside can connect.
- Never enter real passwords, names or personal data into Juice Shop. Nothing in it is private or secure.
- Do not expose VM 2's WAF port to a shared or production network.

12.2 Safety built into the project

Control	How it works
No published port on the shop	The shop is reachable only through the WAF; the health check reports a failure otherwise.
Local-only by default	The WAF listens on `127.0.0.1` until LAN access is chosen.
Acknowledgement	The first Setup requires typing `I UNDERSTAND` after reading the warning.
Container hardening	`no-new-privileges` on both containers.
Contained demo traffic	The guided demo sends requests only to the WAF on VM 2 itself.

12.3 What to know about the script itself

- It runs as **root** (through sudo) because it installs software and controls Docker. Read the source before running it on a machine you care about.
- If Docker is missing it may download and run the official installer from `get.docker.com`, and may download the Compose plugin from GitHub. These are used only as fallbacks when the normal package route fails. The Compose download uses GitHub's `latest` release and is not version-pinned.
- The shop image uses the `latest` tag and is not pinned.
- The script opens a firewall port only when `ufw` is active and LAN access is chosen.

12.4 Cleaning up

After the session: **5 Tools, 4 Reset / uninstall** removes the containers and the lab network; shut down or delete the VMs; if they were exported or snapshotted, treat the snapshots as containing a vulnerable system.

13. Limitations, known issues and items to verify

13.1 Limitations (by design)

Limitation	Explanation
A WAF does not fix application flaws	Business-logic and access-control problems pass through in both modes. This is the point of the last demo row, not a defect.
Only rule-triggering requests are logged	The live view and summary list requests that matched WAF rules. Normal traffic appears only in the access-log totals.
Totals cover the time since the WAF last restarted	The "allowed vs blocked" request counts come from the container's own log; only attack events are preserved across restarts (the journal).
Juice Shop is a Node.js application	Command-injection and `/etc/passwd` payloads are blocked by the WAF with protection ON, but with protection OFF they do not truly execute on the server.
Client-side (DOM) attacks	A WAF only sees HTTP requests; script that runs entirely inside the browser may still run even if the request that carried it is blocked.
One target application	Version 1.0 supports OWASP Juice Shop only.
Linux server only	VM 2 must be Debian/Ubuntu for the automatic installation (other Linux systems use the Docker installer script as a fallback). The host running the hypervisor can be any operating system.
Several network adapters	The cheat-sheet uses the first address that `hostname -I` reports (ignoring 127.x and 172.17.x), which may be the NAT address while the NAT adapter is present.
Images are not version-pinned	The `latest` Juice Shop tag and the `nginx` WAF tag can change; behaviour and rule IDs may change with them.

13.2 Items to verify on your own lab ("To be verified")

These could not be confirmed from the source code alone. Check each once on your real lab and note the result.

#	Item	How to verify
1	The log views show events on a real WAF (audit record format, rule IDs).	Send one attack, open **3, 1** and **3, 2**; if empty open **3, 3** and compare.
2	Juice Shop's answer to the basket-tampering request (200 or an error), and that it is not blocked in ON mode.	Run the guided demo; read the last row; run it a second time.
3	Exact HTTP answers of Juice Shop for the search-based requests with protection OFF.	Run the commands in section 10.4 with protection OFF.
4	How quickly `nikto` and `sqlmap` are blocked with protection ON (rule 913100 for the `sqlmap` agent, plus request volume).	Run both against the WAF with protection ON and watch the live view.
5	The mirror image names `ghcr.io/juice-shop/juice-shop` and `ghcr.io/coreruleset/modsecurity-crs:nginx` exist.	Run `docker pull` for each name from a machine with internet.
6	Resource sizes (4 GB RAM, 20 GB disk for VM 2) are comfortable for your hardware.	Watch `free -h` and `df -h` during Setup and the demo.
7	Behaviour at paranoia levels 2 and 3 (which harmless searches are blocked).	Rehearse Act 7 and note a reliable example.
8	Which address the cheat-sheet prints when VM 2 has two adapters.	Run **5, 1** before and after removing the NAT adapter.

14. Glossary and FAQ

14.1 Glossary

Term	Meaning
Anomaly score	A running total of points that matching rules add to a request. Reaching the threshold marks the request as an attack.
Audit log	ModSecurity's detailed record of a flagged request: who sent it, what was requested, which rules matched, and the answer.
Bridged / NAT / Host-only network	Virtual network types. Bridged puts a VM on your real LAN; NAT lets it reach the internet through the host; Host-only connects the VMs to each other and the host only.
Business-logic flaw	A weakness in how the application behaves (for example no check that a basket belongs to the user), not in how requests look.
Command injection	Tricking an application into running an operating-system command supplied by the attacker.
Container / image	A running isolated program / the template it is created from.
CRS (OWASP Core Rule Set)	The free rule collection used by ModSecurity to recognise attacks. Rules have numeric IDs grouped by attack family.
DetectionOnly	ModSecurity mode that logs matches but does not block (protection OFF in this project).
Docker / Docker Compose	Software that runs containers / starts several containers from one description file.
False positive	A legitimate request wrongly blocked.
HTTP 200 / 403 / 502	200 = success; 403 = forbidden (here: blocked by the WAF); 502 = the proxy could not reach the application behind it.
Hypervisor	Software that runs virtual machines (VirtualBox, VMware, Proxmox, KVM).
Juice Shop	OWASP's deliberately vulnerable web shop used as the target.
Kali Linux	A Linux distribution that ships with security-testing tools; used for the attacker VM.
ModSecurity	The open-source WAF engine, here embedded in nginx.
nginx	A web server and reverse proxy; it hosts ModSecurity in the WAF container.
nikto / sqlmap / curl	A web-server scanner / an automatic SQL-injection tool / a command-line HTTP client.
Paranoia level	CRS strictness from 1 (fewest false alarms) to 4 (strictest). This project offers 1 to 3.
Path traversal	Using `../` sequences to reach files outside the intended folder.
Published port	A container port made reachable from outside the machine. The shop has none; the WAF has one.
Reverse proxy	A server that receives requests for another server and forwards them.
SQL injection	Smuggling database commands into a request so the application runs them.
tarball (.tar)	A single archive file; here used to carry a Docker image to an offline machine.
ufw	Ubuntu's simple firewall front-end.
unique_id	A code ModSecurity gives every request; it lets the script match the same request across different log lines.
WAF	Web Application Firewall: inspects web requests for attack patterns.
XSS (cross-site scripting)	Injecting script into a page so it runs in other users' browsers.

14.2 Frequently asked questions

Q: Is it safe to run Juice Shop?

Yes, on an isolated network. It is intentionally vulnerable and contains no real data; the danger is only in exposing it to a real network.

Q: Can I use this to protect a real website?

No. This is a teaching lab. Production WAFs need tuning, monitoring, TLS handling and high availability.

Q: Why does the attacker get 403 and not a different error?

403 Forbidden is how this WAF refuses a request that reached the blocking threshold.

Q: Does "passed through" mean the attack succeeded?

It means the request reached the application. Whether the attack truly works depends on the application (for example the SQL login bypass works; the command injection does not execute in a Node.js app).

Q: Why does switching the mode restart the WAF?

The rule engine setting is read when the container starts, so the container must be recreated. Juice Shop is not restarted.

Q: Can I run attacker and server on one machine?

Yes: answer "this machine only" in Setup and attack `http://127.0.0.1:8080`. It is less convincing than two machines and has no network isolation story.

Q: Can I attack Juice Shop on port 3000?

No, by design. That is the point of the architecture.

Q: How do I update or reinstall?

5 Tools, 4 Reset / uninstall, then **1 Setup**. Images are kept, so it is quick.

Q: Where are the settings stored?

In `/opt/waf_demo/state.json`; see Appendix A.

Q: Can another application be added?

Not in version 1.0. A target would need its own compose service, internal port and demo requests.

Appendix A — File and folder layout

```
/opt/waf_demo/                  (or ~/.waf_demo when not root; override with WAF_DEMO_HOME)
├── state.json                  saved settings (mode, paranoia, bind, port, images, ...)
├── docker-compose.yml          generated by the script; do not edit by hand
├── events.jsonl                saved WAF events (survive WAF restarts; cleared by "Clear logs")
├── error.log                   details of the last unexpected error (only if one occurred)
├── get-docker.sh               only if the official Docker installer was needed
└── images/                     optional: *.tar image archives for offline installation
```

Container / network	Name
WAF container	`wafdemo-waf`
Shop container	`wafdemo-juice-shop`
Docker Compose project	`wafdemo`
Private network	`wafdemo-net`

Appendix B — The generated compose file and its variables

This is the exact template embedded in the script. The `${...}` values are filled from `state.json` each time Compose is called.

```
# wafdemo - generated by waf_demo_manager.py. Do not edit by hand; use the menu.
services:
  juice-shop:
    image: ${LAB_JUICE_IMAGE:-bkimminich/juice-shop:latest}
    container_name: wafdemo-juice-shop
    # Deliberately NO 'ports:' entry - the app is reachable ONLY through the WAF.
    security_opt:
      - no-new-privileges:true
    networks: [wafnet]
    restart: unless-stopped
  waf:
    image: ${LAB_WAF_IMAGE:-owasp/modsecurity-crs:nginx}
    container_name: wafdemo-waf
    ports:
      - "${LAB_BIND:-127.0.0.1}:${LAB_WAF_PORT:-8080}:8080"
    environment:
      - BACKEND=http://juice-shop:3000
      - PARANOIA=${LAB_PARANOIA:-1}
      - MODSEC_RULE_ENGINE=${LAB_RULE_ENGINE:-DetectionOnly}
      - ANOMALY_INBOUND=5
      - ANOMALY_OUTBOUND=4
      - MODSEC_AUDIT_ENGINE=RelevantOnly
      - MODSEC_AUDIT_LOG=/dev/stdout
      - MODSEC_AUDIT_LOG_FORMAT=JSON
      - TZ=${LAB_TZ:-UTC}
    security_opt:
      - no-new-privileges:true
    depends_on: [juice-shop]
    networks: [wafnet]
    restart: unless-stopped
networks:
  wafnet:
    name: wafdemo-net
```

Variable	Source	Values
`LAB_BIND`	`bind` in state	`127.0.0.1` or `0.0.0.0`
`LAB_WAF_PORT`	`waf_port` in state	8080 or the next free port
`LAB_PARANOIA`	`paranoia` in state	1, 2 or 3
`LAB_RULE_ENGINE`	`rule_engine` in state	`DetectionOnly` or `On`
`LAB_WAF_IMAGE`	`images.waf` in state	WAF image name that was actually found
`LAB_JUICE_IMAGE`	`images.juice` in state	Juice Shop image name that was actually found
`LAB_TZ`	not set by the script	defaults to `UTC`

Appendix C — Complete menu map

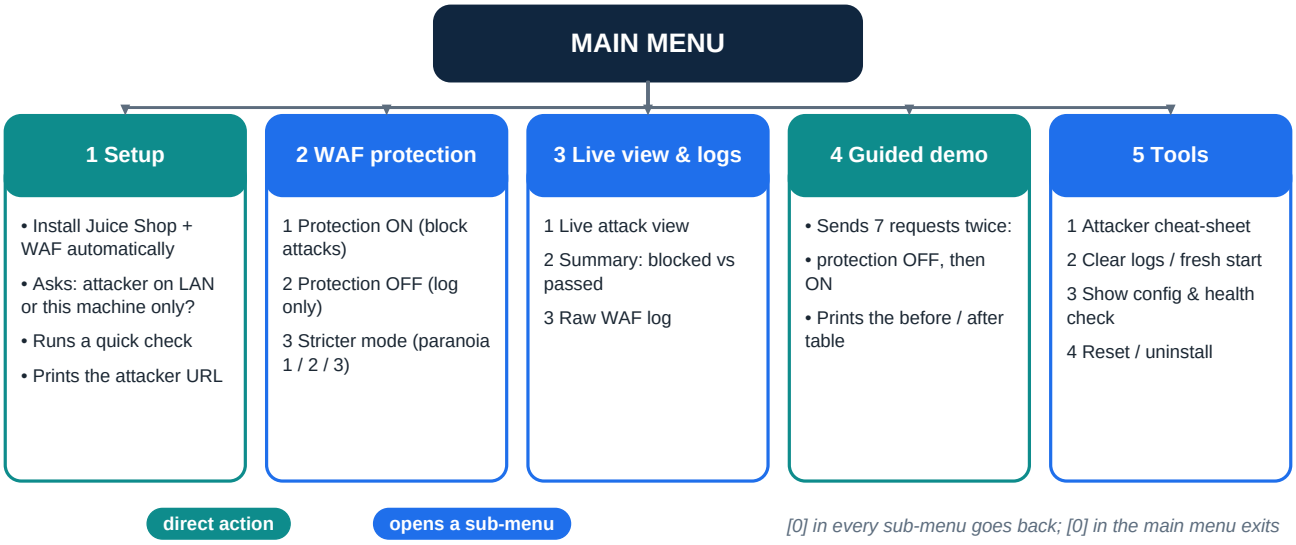


Figure 13. *All menu choices of version 1.0.*

Keyboard: type the number and press Enter. **0** goes back (or exits from the main menu). **Ctrl+C** stops a live view and returns to the menu.

Appendix D — Attack types and CRS rule families

The script names an attack by the first three digits of the CRS rule ID (the first match in this priority order). If no family matches, it shows "Suspicious request".

Rule IDs	Shown as	Meaning
942xxx	SQL injection	Attempts to run database commands through a request
941xxx	Cross-site scripting	Script injected into a page
932xxx	Command injection	Operating-system commands in a request
930xxx	Path traversal	Reaching files outside the allowed folder
931xxx	Remote file inclusion	Making the server fetch a remote file
934xxx	Node.js injection	Server-side JavaScript / Node.js injection
933xxx	PHP injection	PHP code injection
944xxx	Java attack	Java-specific attacks
943xxx	Session fixation	Forcing a known session ID
913xxx	Scanner / attack tool	Known scanner or attack-tool signature
921xxx	HTTP request smuggling	Malformed or split HTTP requests
920xxx	Protocol violation	Request violates HTTP rules
911xxx	Method not allowed	HTTP method not allowed
910xxx	Bad IP reputation	Request from a bad IP address
912xxx	DoS protection	Denial-of-service protection

Correlation rules (ignored for naming)	Role
949110	Reports that the inbound anomaly threshold was exceeded (the blocking decision)
959100, 980130, 980170	Summary and correlation messages

Appendix E — Post-demo cleanup checklist

	Step
[ ]	Save anything you want to keep (for example screenshots of the summary).
[ ]	VM 2: **5 Tools, 2 Clear logs** if you will run another session, or **5 Tools, 4 Reset / uninstall** to remove the lab.
[ ]	Set protection back to the state you want to start the next session in.
[ ]	Shut down both VMs.
[ ]	Delete VM snapshots or exported appliances that you will not reuse (they contain a vulnerable system).
[ ]	Confirm that no VM has a Bridged or NAT adapter attached.

Appendix F — Pre-demo checklist (printable)

	Check	Expected
[ ]	Both VMs start; VM 2 has only the Host-only adapter (NAT removed).	`ip -br addr` on VM 2 shows only the lab address
[ ]	VM 1 can ping VM 2.	`ping -c 3 192.168.56.20` gets replies
[ ]	Menu **5, 3 Show config & health check**.	"Everything looks healthy; Juice Shop is isolated"
[ ]	From VM 1: `curl -m 5 -i http://<server>:3000`.	refused or timeout
[ ]	From VM 1: `curl -i http://<server>:8080/`.	`HTTP/1.1 200 OK`
[ ]	Protection OFF: one attack from VM 1.	answer is not 403; line `PASSED - logged only` in the live view
[ ]	Protection ON: the same attack.	`HTTP/1.1 403 Forbidden`; line `BLOCKED (403)`
[ ]	Guided demo run once.	table with 6 blocked rows and the last row "STILL PASSES"
[ ]	**5, 2 Clear logs** so the audience starts with an empty view.	live view is empty
[ ]	Screens arranged as in Figure 12; terminal on VM 1 has `URL=...` set.	attack left, WAF decision right
[ ]	Terminal text size readable from the back of the room.	

Quick-start card

Ten steps from nothing to a working demo.

1	Create two VMs on one **Host-only** network. VM 2 (Ubuntu/Debian) also gets a temporary **NAT** adapter.
2	Give static addresses, for example **192.168.56.20** (server) and **192.168.56.10** (attacker, Kali). Check with `ping`.
3	Copy `waf_demo_manager.py` to VM 2. `scp waf_demo_manager.py user@192.168.56.20:~/`
4	On VM 2 start the script and choose **1 Setup**. Type `I UNDERSTAND`. Choose **1** (attacker on the LAN). `python3 waf_demo_manager.py`
5	Note the attacker address printed at the end, e.g. `http://192.168.56.20:8080/`.
6	Check isolation from VM 1: port 3000 must fail, port 8080 must answer. `curl -m 5 http://192.168.56.20:3000` / `curl -i http://192.168.56.20:8080/`
7	Remove the NAT adapter from VM 2, restart it, run **5 Tools, 3 Show config & health check**.
8	Demo, part 1: **2 WAF protection, 2 Protection OFF**. From VM 1 run the attacks of **5, 1 Attacker cheat-sheet**. Watch **3, 1 Live attack view**.
9	Demo, part 2: **2, 1 Protection ON**. Repeat the attacks: 403. Then run **4 Guided demo** and explain the last row.
10	Finish with **5, 2 Clear logs** (next session) or **5, 4 Reset / uninstall** (remove the lab).

TIP: Stuck? Run **5 Tools, 3 Show config & health check**. It detects most problems and repairs them. Section 11 explains the rest.